

Real-Time Sales Leaderboard System Design for

Millisecond-Latency, City-to-Global Scale Date: 2025-08-10 UTC

Real-Time Sales Leaderboard System Design for
Millisecond-Latency, City-to-Global Scale Date: 2025-08-10 UTC

One-Pager Summary

Goal Deliver an always-on, low-latency sales leaderboard that aggregates transactions from all branches in real time and serves queries in single to low tens of milliseconds. The platform must be horizontally scalable, cost-aware, secure, multi-tenant, and compliant, with strong correctness guarantees and simple operator ergonomics.

Key Outcomes

End-to-end ingestion to visible rank updates within two seconds p95 or better
Millisecond-level query latency at p95 for top N, rank, and score lookups Strong correctness with idempotent processing and exactly-once effective semantics Multi-tenant isolation by organization and region with per-tenant budgets and controls Mature CI or CD and observability with canary releases and automatic rollback

Primary Assumptions

Branches ten thousand to one hundred thousand across regions Sellers one hundred thousand to five million identities depending on tenant size Peak ingest five thousand to fifty thousand sales events per second per region Leaderboards required per branch, per region, per product line, and global Read QPS five thousand to fifty thousand across rank endpoints during peaks Query response target p95 under ten milliseconds from the edge for hot sets

Functional Requirements

Record sales events with seller, branch, tenant, amount, currency, product, timestamp Expose leaderboards by scope such as branch, city, region, and global with filters Support top N queries, rank of a given seller, score of a seller, and around me slices Provide time windowed boards such as today, week to date, month to date, and rolling windows Support currency normalization and optional weighting rules Offer administrative controls for resets, eligibility rules, and fraud flags Streaming exports and webhooks for winners, milestones, and incentive triggers Multi-tenant isolation and per-tenant configuration such as cutoffs and rules

Non-Functional Requirements

Availability public read apis ninety nine point nine five percent or better
Ingest availability and durability ninety nine point nine five percent or better Latency p95 query under ten milliseconds, p99 under twenty five milliseconds End to end ingest to visible update p95 under two seconds Data durability RPO zero for accepted events with multi az; RTO minutes Security and compliance aligned to SOC two Type two, GDPR, and CCPA

Architecture Overview

Ingestion

Public order services emit outbox events or database change capture captures sales rows API gateway authenticates partner or branch systems, applies schema validation and rate limits Kafka or Kinesis receives sales events keyed by tenant and branch to preserve per scope ordering Schema registry manages versioned protobuf or json schema contracts Dead letter topics capture validation failures for triage

Stream Processing

Flink or Kafka Streams performs idempotent aggregation by keys tenant, scope, window, and seller Stateful operators maintain per key counters and rolling windows with timers Aggregation emits increments to leaderboard writers and to a time series store Currency normalization is applied using daily or hourly fx snapshots Serving Layer Redis Cluster sharded by tenant and scope is the hot store for leaderboards Zsets hold scores per leaderboard with seller as member and score as cumulative amount Hash maps hold small seller profiles and metadata for hydration and around me slices Read apis hit redis first for rank and top N and hydrate details from a replicated store ClickHouse or BigQuery or Snowflake provide analytical queries and historical rollups

System of Record Orders remain in the merchant or erp database; a normalized write-optimized store holds event copies Option A Postgres or Aurora partitioned by tenant with readonly replicas Option B DynamoDB with composite keys tenant and event id plus gsi for lookups Debezium or transactional outbox ensures exactly-once effective delivery to Kafka

Data Flow Sale generated by branch or partner system Event written to transactional store and outbox or captured by cdc Producer publishes to Kafka with idempotency key event id Stream job aggregates and writes score deltas to redis and persists checkpoints Leaderboards update instantly; analytics sink receives appended facts for history

Technology Stack Edge and auth api gateway waf oauth two point one or openid connect Streaming kafka with schema registry and connect or kinesis Processing apache flink or kafka streams with rocksdb state backends Hot store redis cluster with client side hashing and pipelined commands System of record postgres or aurora or dynamodb as per team and tenancy needs Analytics clickhouse or bigquery or snowflake over s three or gcs with iceberg or parquet Platform kubernetes, grpc between services, rest for public apis, graphql for ui aggregation Ci or cd github actions or gitlab ci, argo cd and argo rollouts

Storage Strategy

Hot path Redis zsets per leaderboard key with daily or rolling window variants One redis hash per seller for small metadata and last seen values Warm path Replica store of normalized seller and branch metadata in postgres or dynamodb Cold path Object storage for raw events in parquet with hourly partitions and iceberg tables for analytics Analytical store ClickHouse for near real time cohort and funnel reporting with materialized views

Scaling Approach Horizontal partitions everywhere by tenant and scope Kafka topics partitioned by tenant and branch provide parallelism and ordering guarantees Flink job parallelism scales with partitions; state is checkpointed to durable storage Redis clusters scale by adding shards; keys use consistent hashing with hash tags per leaderboard key Read api scales behind the gateway with connection pooling and request coalescing

Latency Budgets and Slas Edge plus tls termination under twenty milliseconds p95 Redis operations under three milliseconds p95 local az Hydration from replica store under five milliseconds p95 Service computation and serialization under five milliseconds p95 Total query budget target ten milliseconds p95 from edge Ingest produce under twenty milliseconds; processing and write to redis under one second p95 End to end visibility target under two seconds p95 Availability read apis ninety nine point nine five percent, write apis ninety nine point nine percent

Data Models

Event tenant id, branch id, seller id, order id, event id, product id, amount minor units, currency, timestamp, attributes json Leaderboard Key tenant id, scope type such as branch or region or product, scope id, window type and window id Score seller id, cumulative score numeric, last update ts, count Metadata seller display name and branch, flags, eligibility, country All primary keys include tenant id; scope or window fields are encoded into redis keys and clickhouse partitions

Key Algorithms and Workflows

Idempotency Event id used for dedupe in stream and in redis via set with short time to live or bloom filter to prevent double counting replays Windowing For today week month or rolling windows use tumbling hourly and daily windows with late arrival tolerance and watermarking in flink

Leaderboard Updates Use redis pipeline to apply zadd increments and retrieve rank or score in one round trip Around Me Fetch rank then zrevrange by rank offset to slice neighbors with small hydration lookups Global and Regional Maintain hierarchical leaderboards by emitting updates to multiple scopes derived from branch to region to global mapping Currency Apply fx conversion on stream to a single settlement currency to keep scores comparable

Security Reliability and Compliance Pii minimized store only pseudonymous seller ids and optional display names, no raw card data Encryption tls one point three in transit and kms at rest with per tenant keys when required Access controls rbac and abac, scoped tokens, m tls in mesh, signed webhooks for partners Privacy compliance subject access and deletion endpoints, audit logs, retention policies Reliability multi az everywhere, multi region read replicas and warm standby for the most critical tenants Back pressure and replay through kafka; dead letter and quarantine for bad events

Multi Tenancy Tenant id propagates through every key and event Soft isolation by tenant scoped topics, redis hash tags, and database partitions Hard isolation available as premium with dedicated namespaces and vpcs per tenant or per region Per tenant rate limits, budgets, keys, dashboards, and data encryption boundaries

Ci Cd and Infrastructure as Code Terraform or pulumi and helm charts define clusters, topics, shards, and policies Argo rollouts for progressive delivery with automatic rollback on slo burn or error spikes Schema evolution via registry and contract tests; shadow topics for risky feed changes Online ddl for relational changes with dual read and shadow write windows

Apis and Integrations Public rest endpoints get v one leaderboard returns top n for a scope and window with paging get v one rank returns rank and score for a seller in a scope and window get v one around returns a small window around a seller post v one event ingests a sales event for testing and small tenants Websocket channel pushes delta updates and winner notifications Partner ingestion connector based on signed webhooks or streaming clients Analytics export to s three or gcs and to clickhouse or bigquery Observability and Operational Practices Open telemetry traces and metrics with tail based sampling to capture tail latency Prometheus and grafana golden dashboards for ingestion lag cache hit zset sizes and p ninety five or p ninety nine Structured logs with tenant id request id and user agent and with pii scrubbing Slo budgets per api linked to autoscaling rules for redis and stream consumers Runbooks for shard failover cold cache warm up and topic rebalancing

Capacity Planning and Cost Estimate Reference tenant with two thousand branches and two million daily events Kafka three to six brokers m five or r six g class with twenty four to sixty partitions Flink parallelism thirty two with checkpoint five seconds and high or low watermarks Redis cluster six shards r six g large class with headroom to double Replica store two to three read replicas small to medium instances Rough monthly infra cost order of magnitude twenty thousand to forty five thousand depending on traffic and analytics scans

Testing Strategy and Failure Drills

Unit and property based tests for ranking arithmetic and window boundaries Contract tests for schema evolution and partner connectors Load tests that focus on redis zset hot keys and bursty writes Soak tests overnight to catch memory fragmentation and state growth in flink Chaos drills for broker loss redis shard failover and cold cache recovery Game days for az isolation and read only regional failover

Phased Build Plan

Mvp three months single tenant single region redis plus postgres or dynamodb basic apis top n and rank and around me websocket updates and csv export to object storage Phase two six to nine months multi tenant multi region flink aggregation currency normalization advanced around me and fraud flags clickhouse analytics and canary delivery Phase three twelve months global control plane premium hard isolation per tenant ml based anomaly detection incentive engines and tighter slos to single digit milliseconds p ninety five

Key Choices Trade Offs Risks and Mitigations Redis zsets provide unmatched latency but require careful shard sizing and memory hygiene mitigate with compact structs short keys and periodic defragmentation Postgres plus cdc offers rich consistency but higher ops cost dynamodb simplifies ops but moves

complexity to key design choose based on team skill and query shape Exactly once semantics are difficult across boundaries the effective approach uses outbox idempotency keys and idempotent redis increments Currency conversion introduces complexity standardize on one internal currency with clear fx snapshot governance Hot partitions for top sellers can skew load mitigate with batching pipelining and careful key hashing